AMSS 2026 — Lecture 4: Class Diagrams in the AI Loop

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from behaviour (Week 3) to structure
2. Live demo: drive AI to a class diagram
3. Core notation: the reading floor
4. How AI gets class diagrams wrong
5. Reading critically: the Critique drill
6. Bridge to Week 5 + Lab 2

Recap: Architect and Critic

- **Architect / director:** drive AI through the software development lifecycle (SDLC).
- **Critic / reviewer:** read AI's output and name what's wrong or missing.

In Week 3 you critiqued AI-written tests. Today: the same loop, on a diagram.

From Behaviour to Structure

- **Week 3 pinned what the system does** — a test says "a 60-minute rental costs €3.00."
- **Today: what objects realise it** — a `Rental`, a `Bike`, a `User`, a `Payment`, and how they relate.

The class diagram is the central structural artifact.

Four Threads Today

1. The class diagram as the central structural artifact.
2. Driving AI to produce one.
3. Reading it critically — multiplicity, fake associations, missing whole-part.
4. The literacy floor: what you must read & critique cold.

The Literacy Floor: Critique, Rationale, Traceability

From Week 1: in the oral defense, *unaided*, you must demonstrate:

- **Critique** — read & critique AI-generated artifacts on the spot.
- **Rationale** — articulate why you directed AI a certain way.
- **Traceability** — defend the trace across your project.

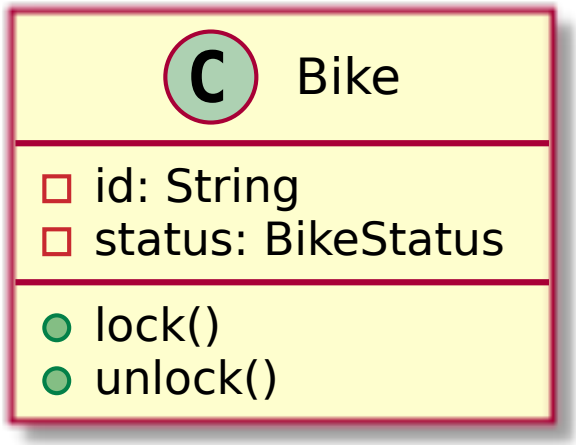
Today is Critique at its sharpest: read an AI-drawn class diagram and name what's wrong.

Demo: Bike-Sharing Class Diagram

Live: drive AI to draw the structure. Watch the multiplicities, the associations, and what's missing.

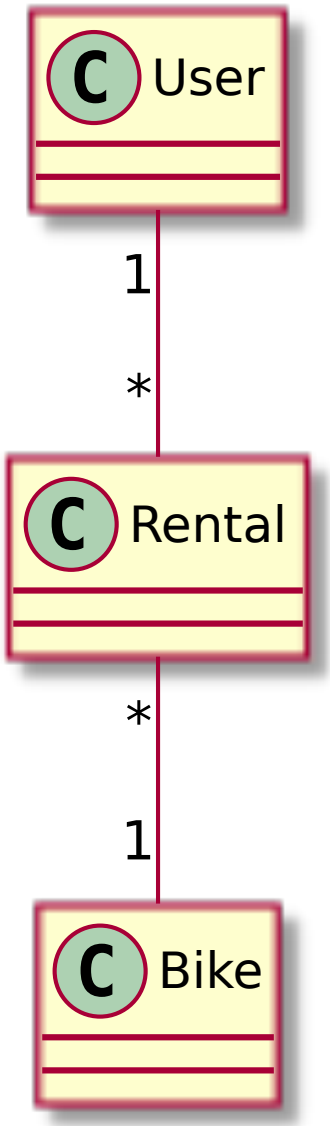
Prompt to AI: *"Generate a UML class diagram (as PlantUML) for the city bike-sharing app: users rent and return bicycles at stations across a city; payment is by app; staff rebalance bikes between stations."*

A Class: Three Compartments



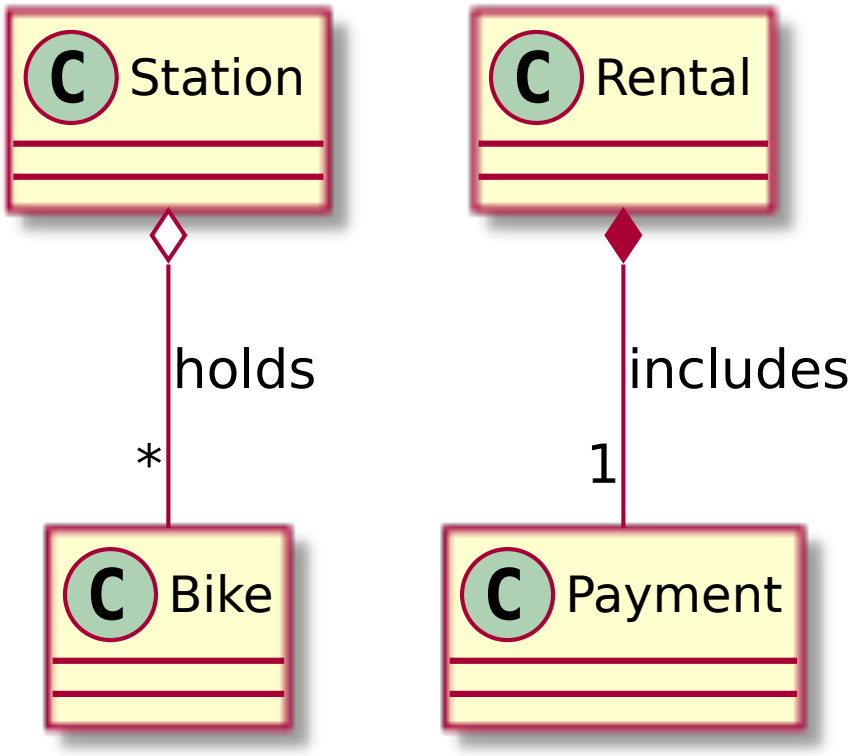
Name, attributes (data), operations (behaviour). – private, + public.

Associations + Multiplicity



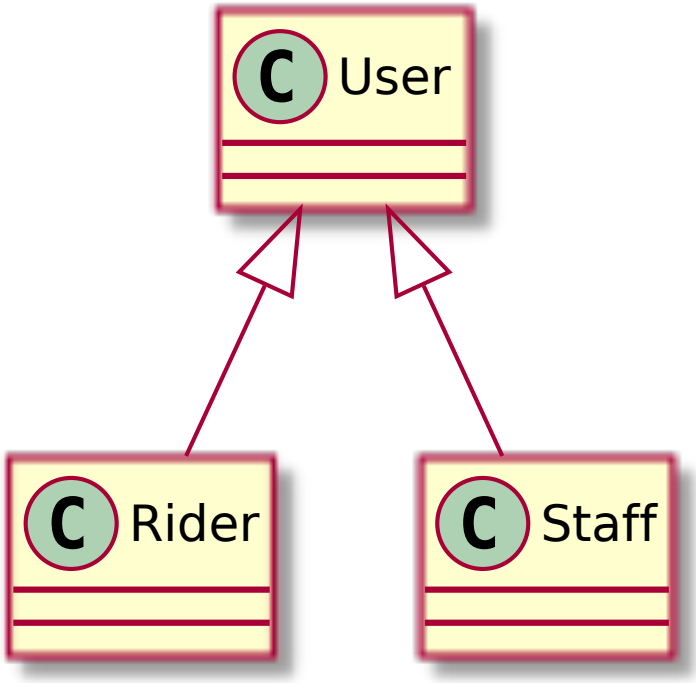
Read it aloud: *one* user has *many* rentals; *many* rentals each reference *one* bike. Multiplicity is where AI most often lies.

Aggregation vs Composition



- **Aggregation** (hollow diamond): a station *holds* bikes; bikes outlive the station.
- **Composition** (filled diamond): a rental *includes* its payment; the payment dies with it.

Generalization (is-a)



A `Rider` *is a* `User`; `Staff` *is a* `User`. The hollow triangle points at the parent.

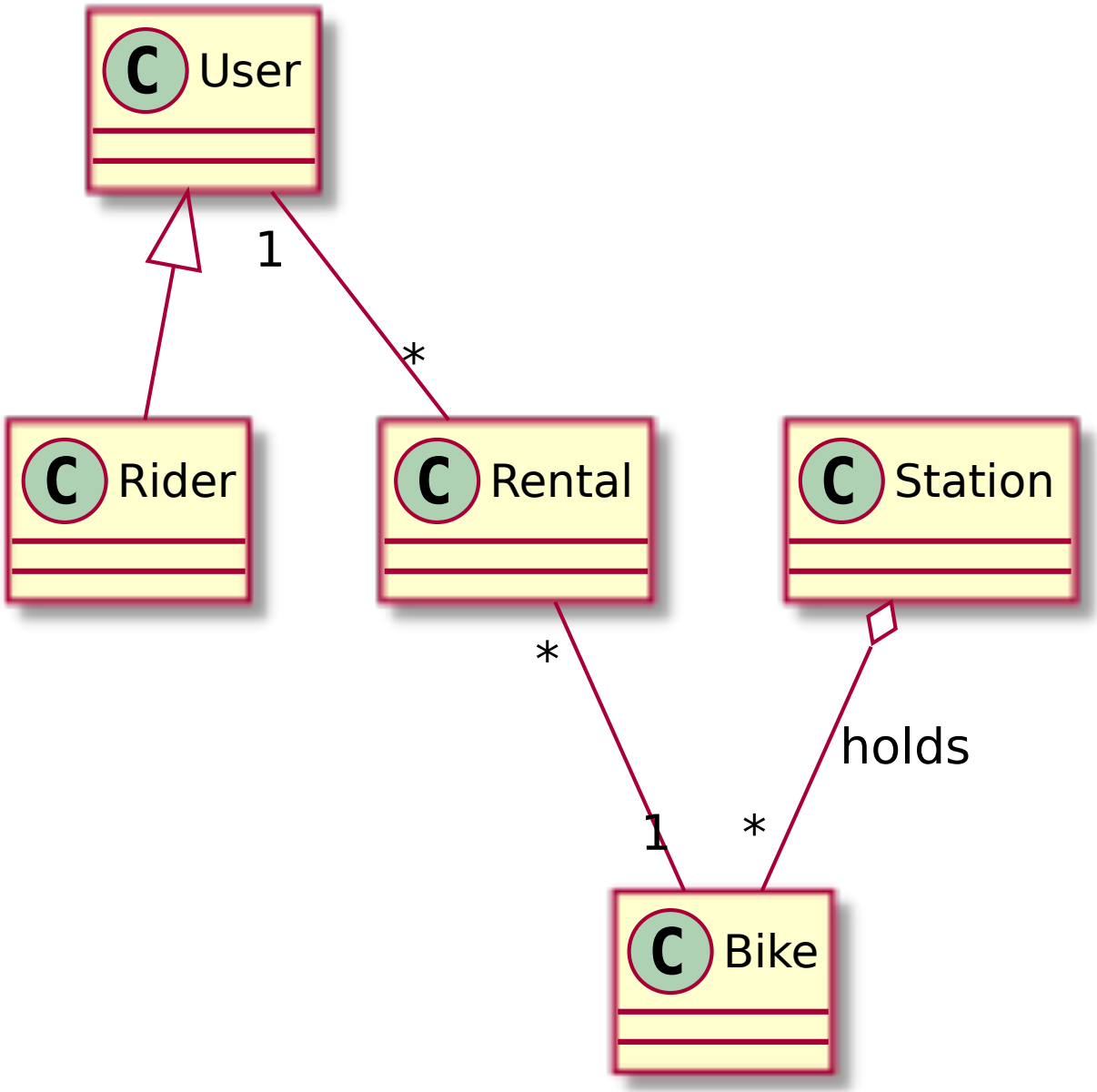
Beyond the Floor

You'll meet these reading AI output — recognise them, but they're not today's focus:

- **Interfaces** — a contract a class implements.
- **Abstract classes** — a partial parent you can't instantiate.
- **Dependency arrows** — "uses" without owning.

Today's reading floor is the four above: class, association + multiplicity, aggregation/composition, generalization.

The Reading Floor, Together



One small diagram, all four elements. If you can read this, you can

Your Turn: Read It Aloud

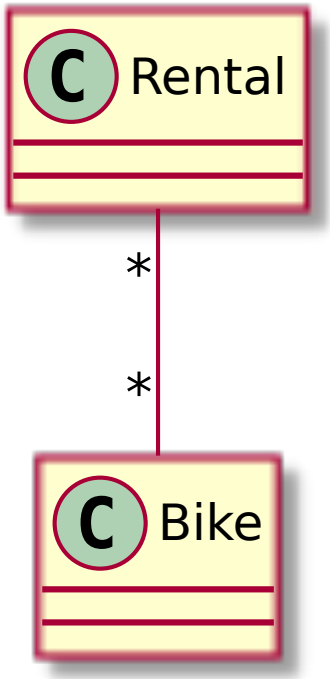
Read this diagram aloud to your neighbour (60s): what does each multiplicity say? Spot anything that can't be right.

AI Draws Plausible Structure — and Gets It Wrong

A class diagram can look professional and still misrepresent the domain. The critic question:

> *"Does this structure match the domain — or just look like a diagram?"*

Defect #1: Wrong Multiplicity



Many-to-many? One rental is *one* bike. **Critique:** *"Read it aloud — can a single rental involve many bikes? Fix the number."*

Defect #2: Fake / Decorative Association

A line between `User` and `Station` with no label, no verb, no meaning.

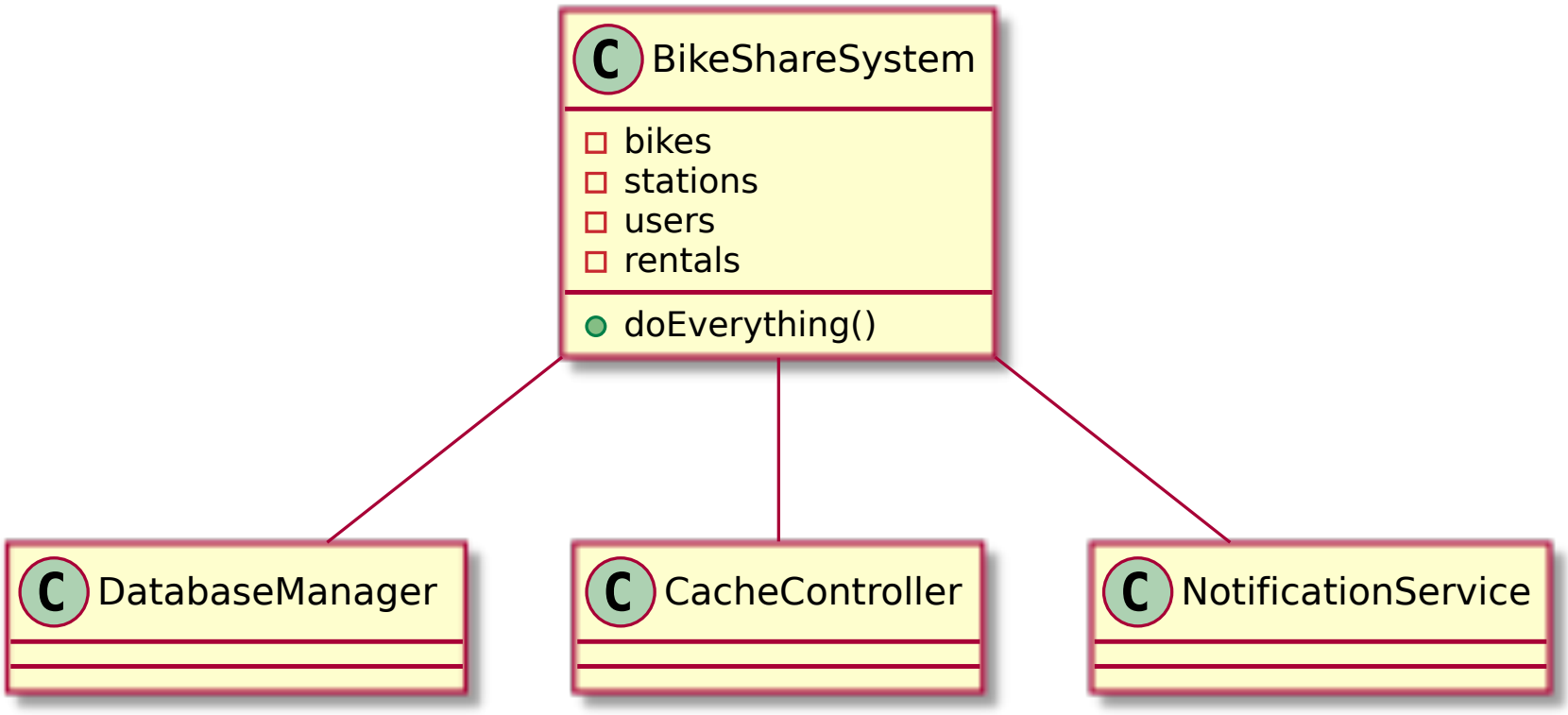
Critique: *"What does this association mean? Name the verb. If you can't, the line shouldn't be there."*

Defect #3: Missing Aggregation

`Station -- Bike` as a plain line hides that a station *holds* bikes — a whole-part relationship.

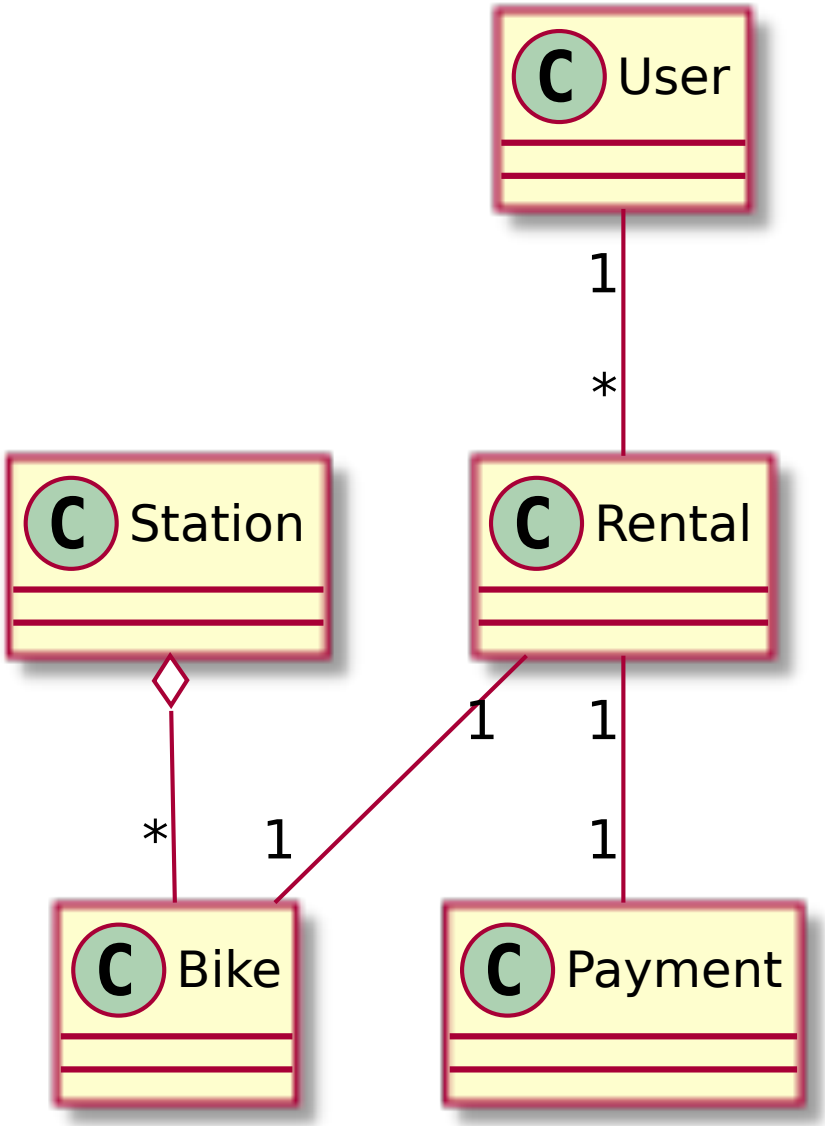
Critique: *"Is this a plain link or a whole-part? Why isn't it an aggregation?"*

AI Over-Models



A **god class** plus **invented infrastructure** (`DatabaseManager`, `CacheController`) — implementation, not domain.

The Critic Simplifies



Same domain, only the classes that earn their place. **You saw defect #1 (multiplicity) live in the demo.**

How to Read an AI Class Diagram

A fixed order, every time:

1. Are the classes real **domain** concepts? (or invented infrastructure)
2. Are the **multiplicities** right? (read each aloud)
3. Does each **association** name a real relationship?
4. Are **whole-part** relationships captured? (aggregation/composition)

The Critique Loop on Structure

read -> name the defects -> re-prompt with domain constraints -> re-read.

Same architect-and-critic loop as Week 2 (requirements) and Week 3 (tests) — now on a diagram.

Apply It to Your Own Output

This week's lab (Lab 2): you drive AI to a class diagram from a 1-page spec, then run *this* read-order against what it draws.

The critique log is the evidence — what you caught and how you fixed it.

The Human Decides What Matches the Domain

AI can draw plausible structure forever. Deciding whether it matches the *domain* — that's the architect-and-critic call.

It's what the oral defense checks, and AI can't make it for you.

This Week's Lab: Lab 2

- Drive AI to produce a class diagram from a 1-page spec.
- Iterate at least twice; log what changed and why.
- Deliverable: the PlantUML diagram + a critique log (1 page max).
- Commit to the course lab repo.

Next Week: Other Structural Views (Week 5)

One class diagram isn't the whole structure.

Week 5: object, package, component, and deployment views — and where AI over- or under-decomposes at the architecture level.

Critique, Rationale, Traceability — The Through-Line

Today you drilled:

- **Critique** — read an AI class diagram and named its defects on the spot.
- **Traceability** — the class is a structural node in the trace: requirement -> use case -> class (the behavioural tail, the test, came in Week 3).

Rationale (why you directed AI a certain way) lands in your project narrative.

That's It For Today

- Next lecture (Week 5): other structural views.
- This week's lab (Lab 2): drive AI to a class diagram, then critique it.

Questions?